Decision Trees

Machine Learning with Python — Lecture 1, Session 3 (Oct 19)

EUGLOH — *Problem Solving Using Open-Source Languages; R and Python*

University of Novi Sad

Today's session

- **Motivation** — decisions everywhere; every good question reduces uncertainty
- **Building blocks** — roots, nodes, branches, leaves
- **The splitting criterion** — entropy and Gini impurity
- **CART** — growing a tree greedily; a split computed **by hand**
- **Interpretability** — reading a tree as rules, and where it breaks down
- **Practical aspects** — overfitting, pruning, complexity

Session 3 of 4 today.

Part 1 — Decision trees

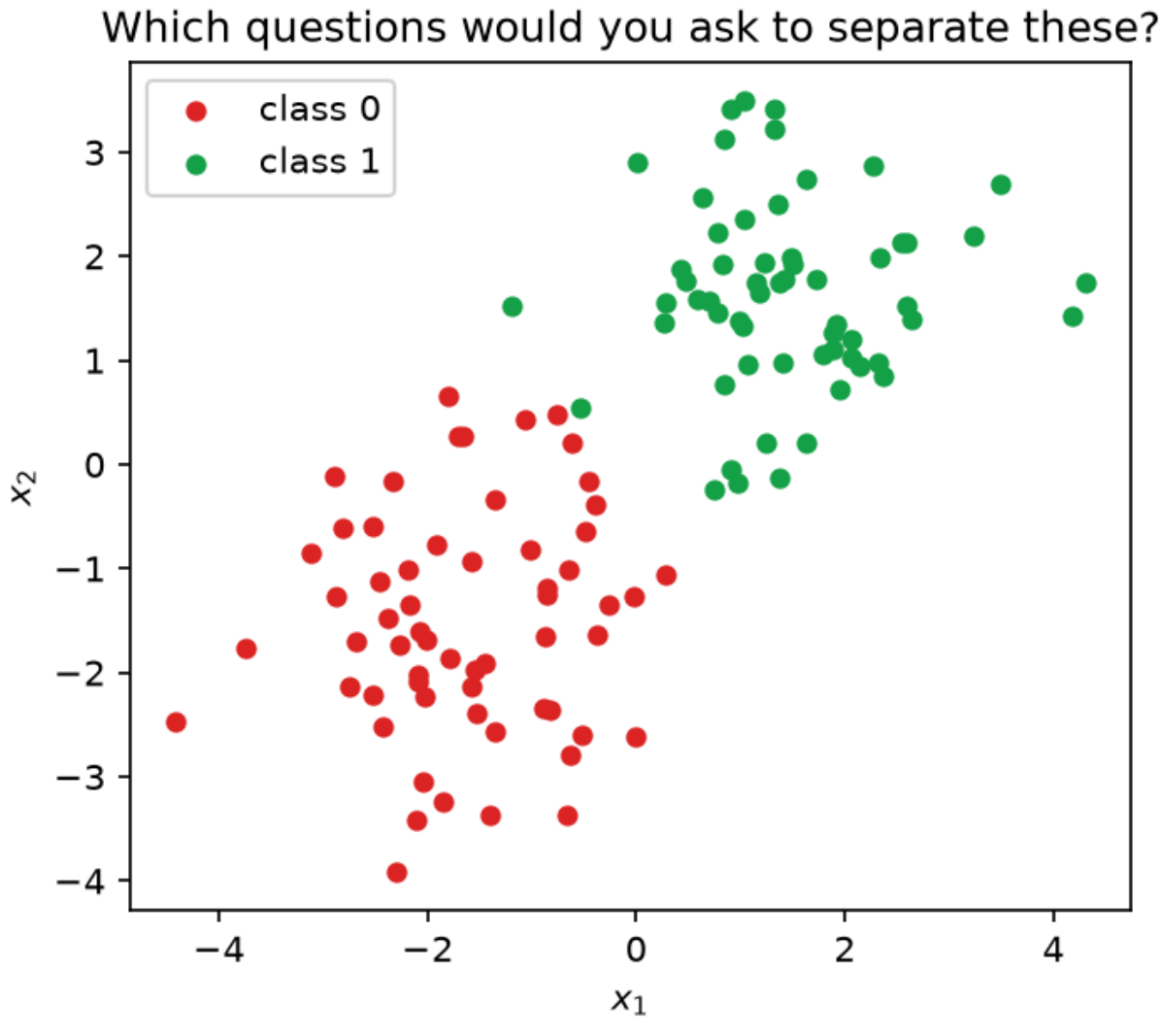
Motivation

Our lives are guided (mostly, but not exclusively) by **decisions**:

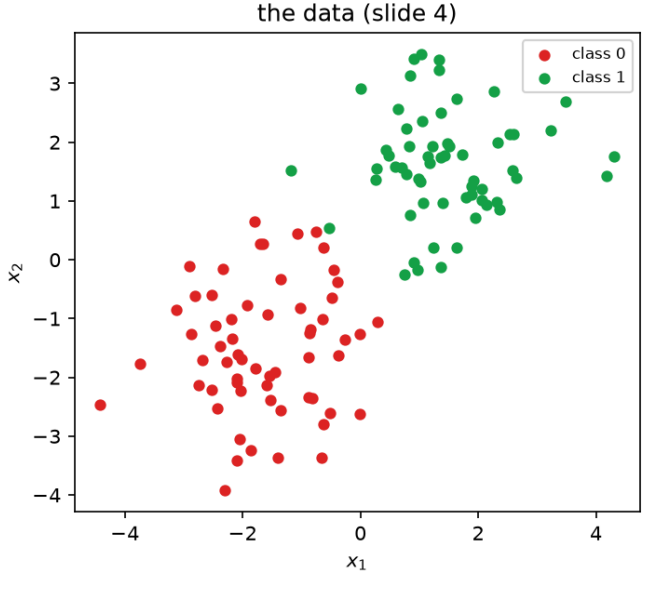
- *"Is it raining?"* → take the umbrella or not
- *"Is the tumor larger than 2 cm?"* → follow-up test or not

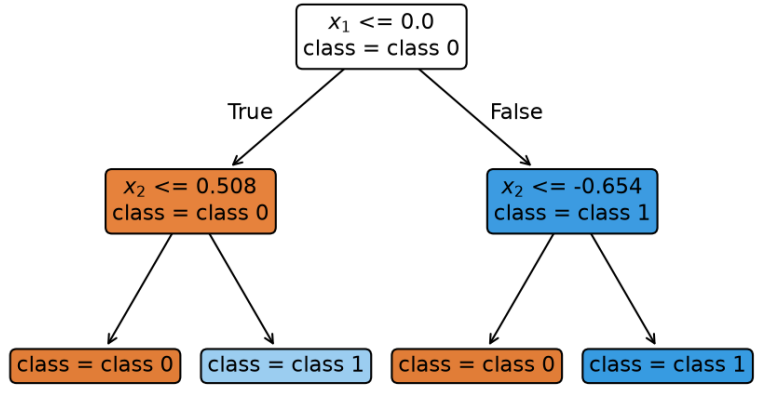
Every question we ask **reduces our uncertainty**; when we reach an answer, the uncertainty is zero. Some questions matter more than others — a good question resolves a lot of uncertainty at once.

A decision tree formalises this: **a sequence of questions, asked in the order that reduces uncertainty fastest.**


Which questions would you ask to separate these?
class 0
class 1
3
2
1
0
−1
−2
−3
−4
x_2
−4
−2
0
2
4
x_1

Fitted to **slide 4's data** (left), here is the tree it grows (right). The first question is the **root**; every box that asks a question is a **node**; **branches** carry the yes/no answer; and each final box is a **leaf** that predicts its majority class. Inference = start at the root, answer the questions, read the leaf.





Building blocks

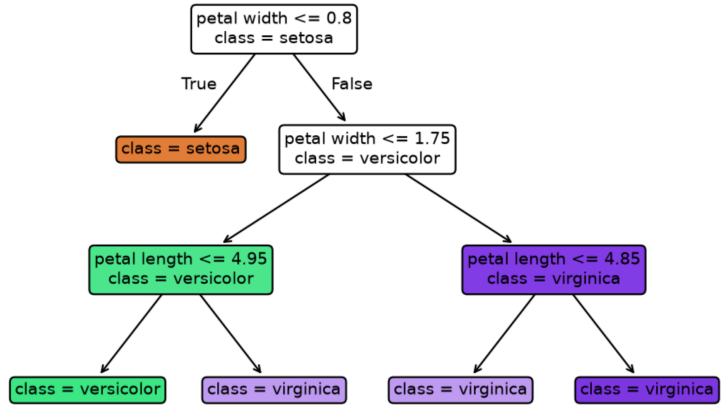
A tree is a simple and powerful framework to navigate through choices:

- **Root** — the first question (sees the whole dataset)
- **Node** — one question about one feature, e.g. $x_1 > 2.3$?
- **Branch** — the answers (yes / no)
- **Leaf** — a final region with a prediction (the majority class)
- **Depth** — the number of questions from root to leaf

Growing a tree: start at the root → ask a question at a node → **pick the best question** → branch out and repeat until termination.

It can do **classification** (spam / not spam) and **regression** (housing prices). Inference is a walk from root to leaf — fully **interpretable**: you can read the model as a flowchart.

A deeper tree (iris, depth 3) — still readable as rules:



The splitting criterion

What makes a question *good*? One that **reduces uncertainty** — so first we need a way to measure how mixed up a group of samples is. Suppose a fraction p of the samples in a node belong to class 1.

Gini impurity (scikit-learn's default, and the one we will use):

$$G(p) = 2\,p\,(1 - p)$$

Entropy (the information-theory version — same idea, a different curve):

$$H(p) = -p\log_2 p - (1-p)\log_2(1-p)$$

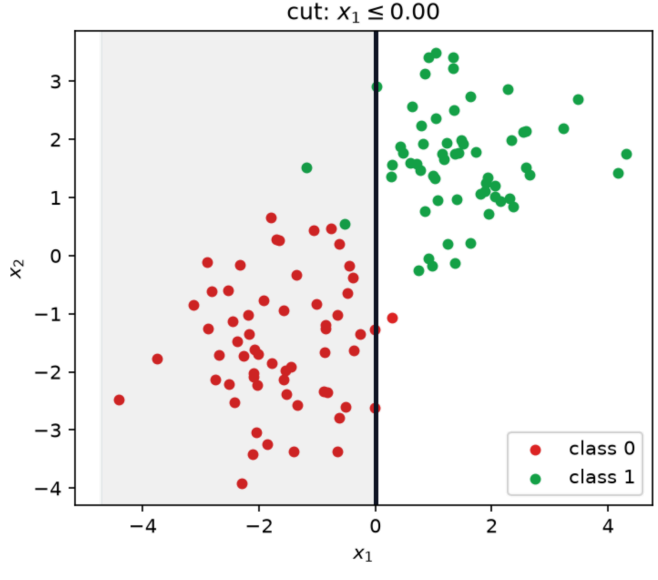
- both are 0 when the node is **pure** (all one class)
- both are **largest** when the node is an even $50/50$ mix ($p = 0.5$)
- a good split makes the two children *purer* than the parent

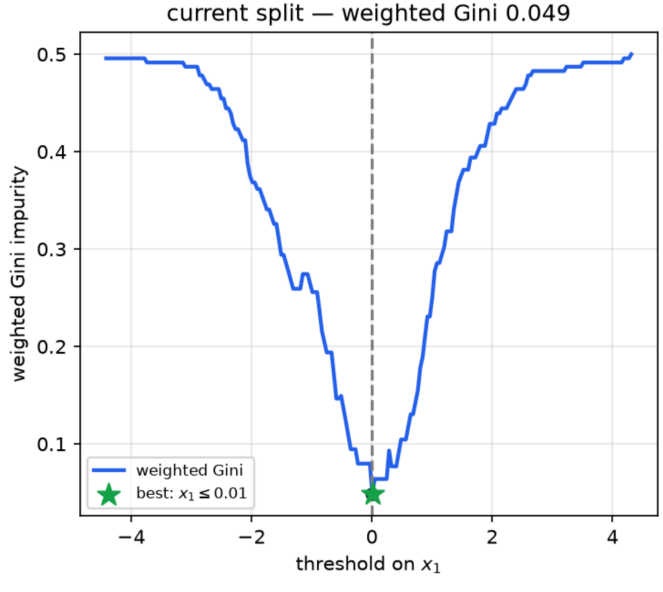
In plain words: one number that answers *"how mixed is this group?"* — 0 means perfectly pure, larger means more mixed.

Interactive demo — how good is a split?

CART grows a tree by trying many splits and keeping the ones that leave the children **purest**. Here is slide 4's data: the slider moves a single cut along x_1. The right panel evaluates *every* candidate cut — the **weighted Gini impurity** of the two children. The lowest point is the split CART would pick.

Split threshold on x_1 0





CART — growing the tree

Classification **A**nd **R**egression **T**rees:

1. Start with **all training samples at the root**
2. At each node, try every feature and every candidate threshold. For each candidate, compute how impure its two children would be, as a **weighted average**:

$$\text{impurity of a split} = \frac{n_L}{n} G(\text{left}) + \frac{n_R}{n} G(\text{right})$$

where n_L and n_R are the numbers of samples going left and right.

3. Pick the split with the **lowest** weighted impurity (greedy!)
4. Recurse on each child — until a **stopping condition** (max depth, pure nodes, min samples per leaf, ...)

The result is a **binary tree**; a leaf predicts its majority class.

Where "greedy" bites — a 2D example

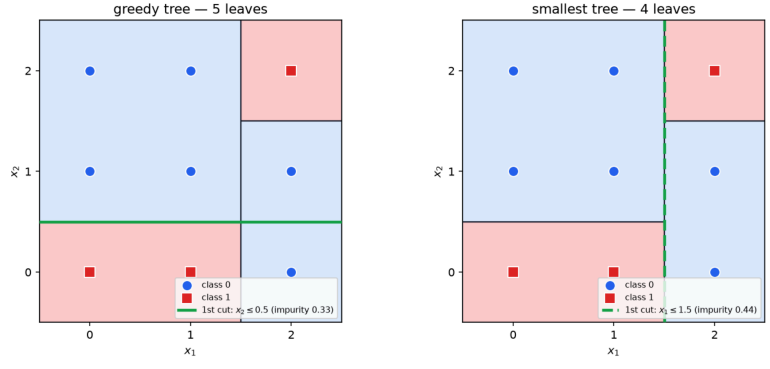
CART never looks ahead: at each node it takes the split with the **lowest immediate impurity**, then repeats. That is **not** the same as building the smallest — or best — tree.

Nine points, two classes. Two candidate **first** cuts:

- **horizontal** $x_2 \leq 0.5$ → immediate impurity **0.33** ← greedy picks this
- **vertical** $x_1 \leq 1.5$ → immediate impurity **0.44**

The horizontal cut looks better *right now*, but it strands the class-1 corner, so the tree CART grows needs **5 leaves**. The "worse-looking" vertical cut leaves each child's class-1 points in a single strip, so the **smallest tree needs only 4 leaves** — from a first cut that was *more* impure. Both classify the data perfectly.

So greedy's myopia is visible in 2D — this is a real nine-point counterexample. It is just rarer and milder on tiny clean data, and becomes the norm once features and label noise multiply.



Worked example — one split, by hand

Six samples, one feature:

x	1	2	3	4	5	6
y	0	0	0	1	0	1

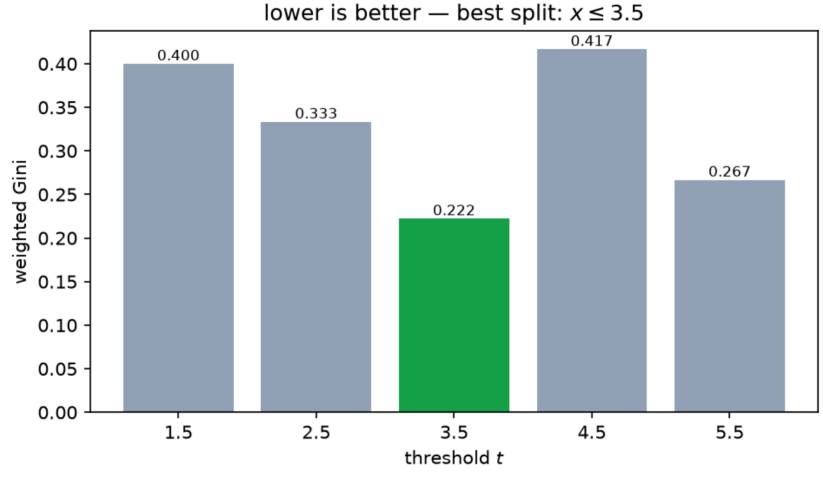
A split sends some samples **left** and the rest **right**. We score it with the **weighted impurity** of the two children:

$$\text{impurity} = \frac{n_L}{n} G_L + \frac{n_R}{n} G_R, \qquad G = 1 - p^2 - (1-p)^2$$

where n_L, n_R are the child sizes, n the total, and p the fraction of class 1 inside a child. Candidate thresholds sit between points: $t \in \{1.5, 2.5, 3.5, 4.5, 5.5\}$.

Try it with pen and paper first — then let the code check you.

split	G(left)	G(right)	weighted
$x \leq 1.5$	0.000	0.480	**0.400**
$x \leq 2.5$	0.000	0.500	**0.333**
$x \leq 3.5$	0.000	0.444	**0.222** ← best
$x \leq 4.5$	0.375	0.500	**0.417**
$x \leq 5.5$	0.320	0.000	**0.267**



The same split, from scikit-learn

```
from sklearn.tree import DecisionTreeClassifier
stump = DecisionTreeClassifier(max_depth=1).fit(x, y)
stump.tree_.threshold[0]   # -> 3.5
```

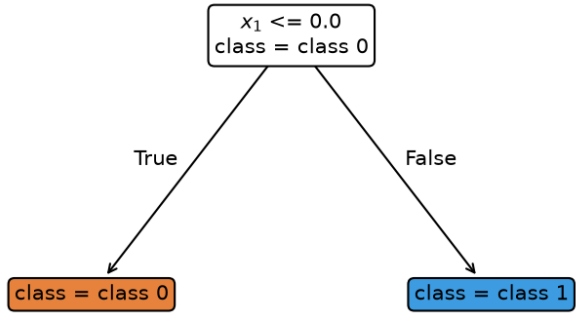
sklearn's root question is $x \leq 3.5$ — exactly the threshold our hand calculation selected. The algorithm tried every candidate and kept the one with the lowest weighted Gini, just as we did.

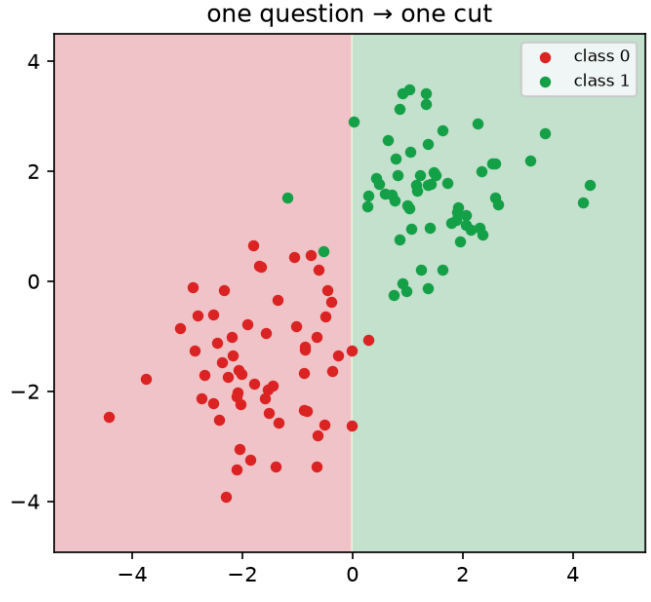
Decision trees in scikit-learn

```
from sklearn.tree import DecisionTreeClassifier, plot_tree
tree = DecisionTreeClassifier(max_depth=2).fit(X, y)
plot_tree(tree, feature_names=[...], class_names=[...], filled=True)
```

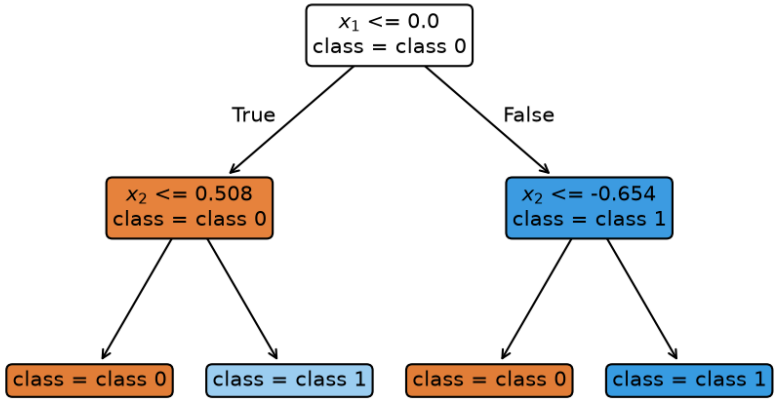
Fitting is instant on small data — and the tree can be *drawn*.

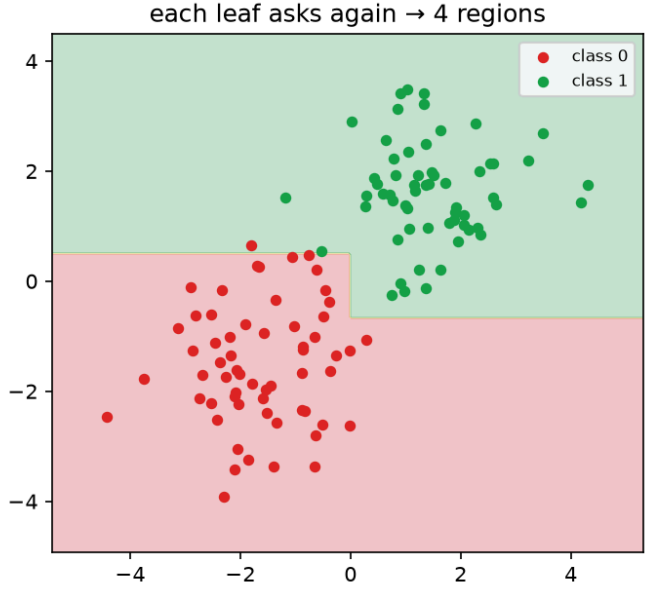
Level 1. The tree asks a *single* question, so it can make exactly **one axis-aligned cut** — the plane is split into two rectangles.





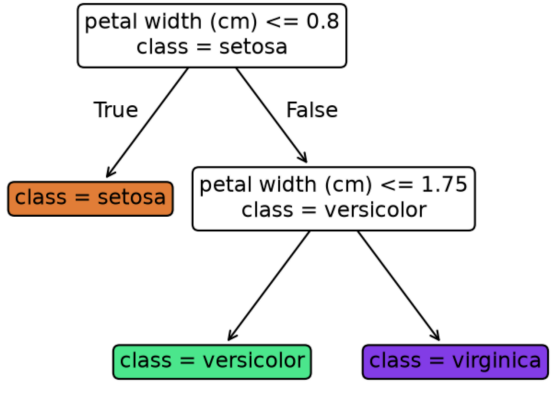
Level 2. Each leaf from level 1 may ask *another* question, carving its region further. Depth 2 gives up to four rectangles — notice every boundary is still **axis-aligned**.

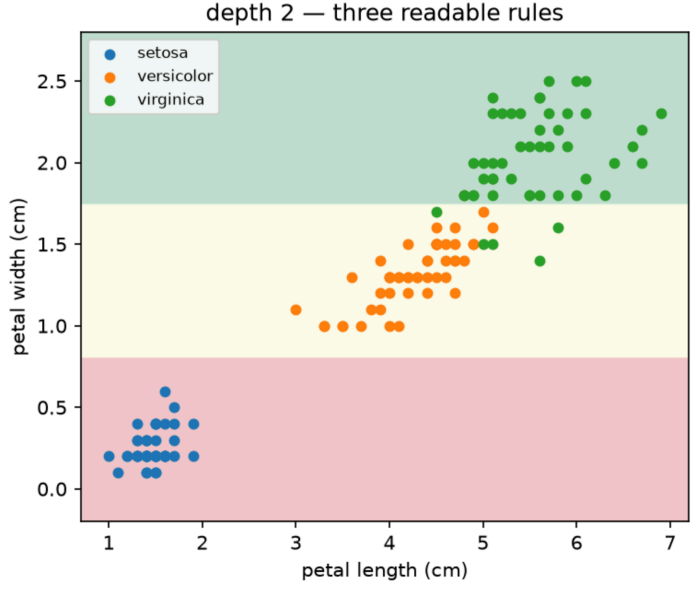




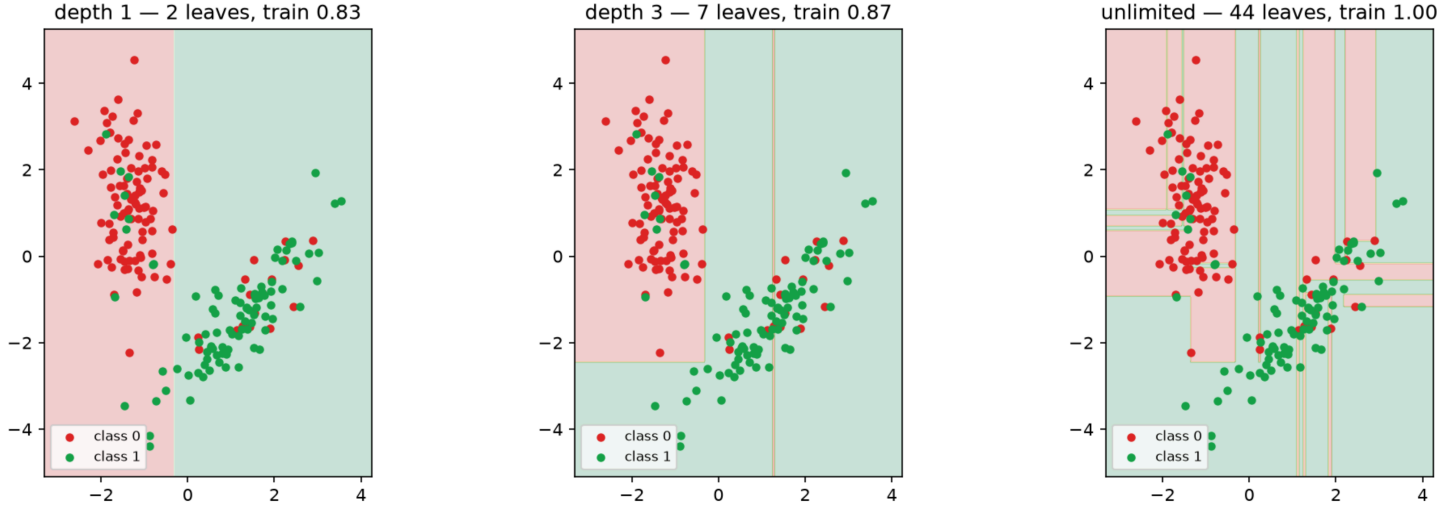
Interpretability. On iris (petal length & width) a shallow tree reads like a field guide — three questions, no code:

- petal width ≤ 0.80 → **setosa**
- $0.80 <$ petal width ≤ 1.75 → **versicolor**
- petal width > 1.75 → **virginica**





Where it breaks down. Same data, deeper trees. The unlimited tree reaches **100% training accuracy** with 44 leaves (and 19 of them hold a single sample). Its boundary is full of **thin slivers** — it splits the *same* feature again and again at almost identical values (e.g. $x_1 \leq -1.35$ followed by $x_1 \leq -1.37$) just to fence off noisy points. Those "rules" fit the noise, not the signal.



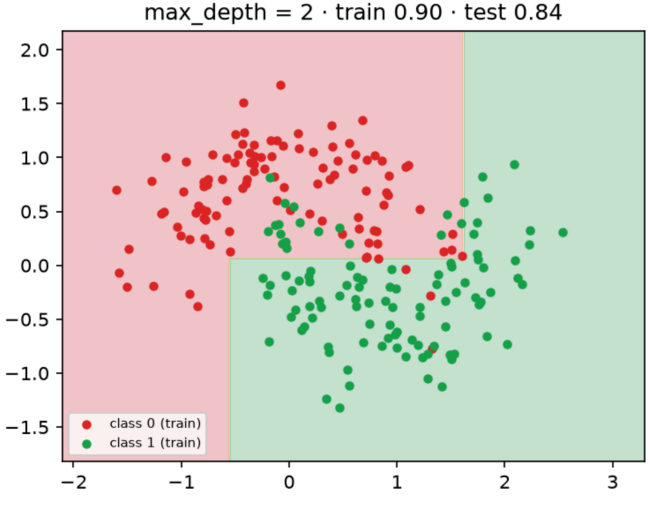
Practical aspects — the dark side: unlimited trees overfit

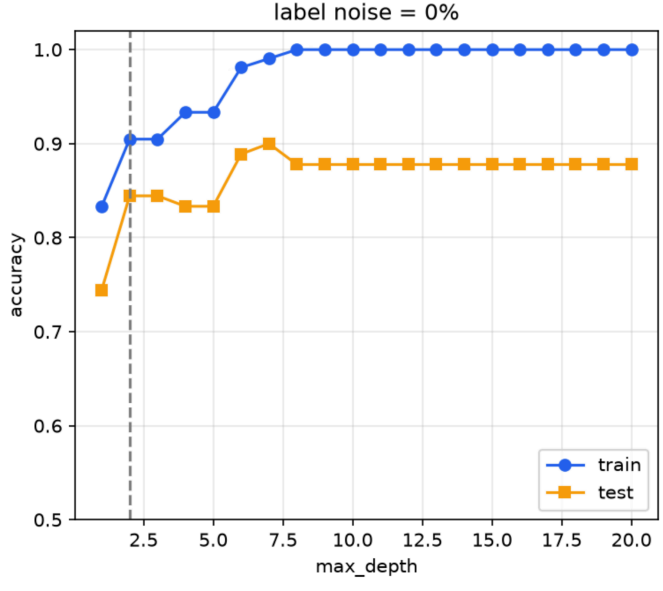
Keep splitting and every training point gets its own rectangle — the tree **memorises noise** and will not generalise.

Why the boundary can look deceptively stable. Deeper trees only *refine* the partition with ever-smaller, **axis-aligned rectangles**. When the true boundary is smooth and the labels are clean, those rectangles simply hug the same curve — the picture barely changes and test accuracy plateaus. Overfitting needs something to memorise: flip the switch to inject **label noise** and watch the test curve peak and then fall as the tree fences off individual mistakes.

max_depth 2

mistake 15% of the training labels





Practical aspects — keeping trees honest

✅ Advantages	❌ Disadvantages
Minimal preprocessing needed	Sensitive to small feature changes
Great for most data types	**Greedy** — no global optimum
Interpretable (a flowchart)	**Overfits** when grown deep
Fast prediction	Training cost $O(m \cdot n \log n)$ — grows gently with data size

Controlling growth:

- Limit it — `max_depth`, `min_samples_leaf`, `min_samples_split`
- **Cost-complexity pruning** — grow a full tree, then gradually remove the subtrees ("weakest links") whose removal increases the error least per leaf removed; use **cross-validation** to pick the best trade-off (`sklearn`: `cost_complexity_pruning_path`, `ccp_alpha`)

(For regression trees the recipe is identical — split by variance reduction instead of Gini, and leaves predict the mean.)

Summary

- **Decision trees** formalise a sequence of uncertainty-reducing questions; roots, nodes, branches, leaves.
- A **good split** minimises **Gini impurity** (or entropy) — computed greedily over every feature and threshold (**CART**).
- Trees are **interpretable** and need little preprocessing — a shallow tree reads as rules — but grown deep they invent **conflicting splits** and **overfit**; control depth, or **prune**.

Next session: **random forests**.